CloudSage — Security, Testing and Quality

Security model, test strategy and engineering quality gates

1. IAM

- Use least privilege
- Separate read-only analysis role from write/execution role
- Avoid long-lived access keys
- Use short-lived credentials/roles where possible
- Review permissions as services are added

2. LLM Security

- Never send credentials to the model
- Treat AWS logs/tags/commit text as untrusted
- Constrain output schema
- Do not allow arbitrary shell commands
- Keep write permissions outside the model
- Log model/version metadata without secrets

3. Terraform Security

- Validate configuration
- Review plan
- Detect unexpected destroys
- Restrict allowed resources/fields
- Require approval
- Keep state protected
- Use remote state with proper locking when the project reaches shared environments

4. Application Security

- Environment-based secrets
- Input validation
- Authentication for UI/API
- Authorization for approvals
- CSRF protection if browser sessions are used
- Rate limiting where exposed
- Dependency scanning

- Container image scanning

5. Unit Tests

- Cost calculations
- Metric aggregation
- Threshold rules
- Candidate generation
- Schema validation
- Policy decisions
- Resource mapping

6. Integration Tests

- AWS collector against a controlled account/test fixtures
- CloudWatch retrieval
- Cost data adapter
- PostgreSQL persistence
- Terraform plan parsing

7. AI Evaluation Tests

- Oversized EC2
- Undersized EC2
- Healthy right-sized EC2
- Missing memory data
- Production resource
- Unexpected Terraform destroy
- No cost data
- Conflicting evidence
- Prompt-injection-like log text

8. End-to-End Test

1. Provision a disposable EC2 test resource with Terraform.
2. Generate/collect controlled utilization.
3. Run CloudSage scan.
4. Verify candidate detection.
5. Verify AI structured output.
6. Verify deterministic savings.
7. Generate Terraform plan.
8. Verify safety gate.
9. Approve in test environment.
10. Apply.

11. Verify health and cost baseline handling.
12. Destroy test environment.

9. Quality Gates

ruff check .
pytest
terraform fmt -check -recursive
terraform validate
terraform plan

10. Definition of Done

A feature is not done merely because it works once. It needs tests, structured errors, logging, documentation, security review, repeatable setup and a clear rollback/failure behavior.